

RAPPORT DE PROJET MACHINE LEARNING

ML Retail Analytics

Optimisation de la Rétention Client par l'Intelligence Artificielle

Présenté par :
Donia Bahloul

Table des matières

1	Introduction et Contexte Métier	2
1.1	Problématique	2
1.2	Objectifs du Projet	2
2	Architecture Technique et Écosystème de Données	2
2.1	Architecture du Projet (MLOps)	2
2.2	Le Dataset Synthétique	2
3	Preprocessing et Feature Engineering	3
3.1	Nettoyage et Imputation	3
3.2	Feature Engineering	3
3.3	La Règle d'Or : Zéro "Data Leakage"	3
4	Modélisation et Algorithmes	3
4.1	Modèle 1 : Segmentation Comportementale (K-Means)	3
4.2	Modèle 2 : Détection du Churn (Random Forest)	3
4.3	Modèle 3 : Prédiction Financière (Régression Lasso)	4
5	Matrice d'Action Business (Insights)	4
6	Déploiement et Opérationnalisation	4
7	Conclusion et Évolutions Futures (MLOps)	5

1. Introduction et Contexte Métier

1.1. Problématique

Dans le secteur ultra-compétitif du e-commerce, le **coût d'acquisition d'un nouveau client est en moyenne 5 à 25 fois plus élevé** que le coût de fidélisation d'un client existant. La rétention client n'est donc pas seulement un simple enjeu marketing : c'est un véritable impératif financier pour assurer la rentabilité à long terme de l'entreprise.

1.2. Objectifs du Projet

Ce projet ambitieux a pour but de construire un pipeline d'intelligence artificielle complet (de bout en bout, ou *end-to-end*) afin de répondre à trois questions métier fondamentales de manière automatisée :

1. **Qui va partir ?** → Prédiction du Churn (attrition).
2. **Combien vont-ils dépenser ?** → Prévision Monétaire (régression financière).
3. **Qui sont nos meilleurs clients ?** → Segmentation Clientèle comportementale.

L'objectif final est de transformer la donnée brute en un système d'aide à la décision proactif, intégré via une interface et une API web.

2. Architecture Technique et Écosystème de Données

2.1. Architecture du Projet (MLOps)

Afin de garantir la maintenabilité et la scalabilité du code, le projet respecte les standards de l'industrie (MLOps) avec une séparation stricte des responsabilités. Le projet est structuré comme suit :

- **data/** : Zone de stockage isolée pour les données brutes et les matrices de features pré-traitées.
- **src/** : Scripts Python modulaires et orientés production (`generate_data.py`, `preprocessing.py`, `train_model.py`).
- **models/** : Espace de persistance pour les modèles sérialisés (`.joblib`), prêts pour l'inférence.
- **notebooks/** : Environnement de recherche pour l'Analyse Exploratoire des Données (EDA).
- **app/** : Application web Flask pour l'interaction utilisateur et la consommation des modèles.

2.2. Le Dataset Synthétique

Pour simuler un environnement professionnel réaliste et exigeant, un dataset synthétique de **1000 clients uniques** comprenant **52 variables initiales** a été généré algorithmiquement. Ce jeu de données se distingue par sa complexité intentionnelle :

- **Déséquilibre de classes massif** : Seulement 18% de clients churners pour 82% de clients fidèles, imitant la rareté des signaux positifs en conditions réelles.
- **Bruit et anomalies** : Injection ciblée de valeurs aberrantes (ex : satisfaction à 99, codes d'erreur système), et de valeurs manquantes (ex : 30% d'âges non renseignés).
- **Richesse des features** : Combinaison de variables RFM (Récence, Fréquence, Montant), de métriques comportementales (temps passé, ratios), et d'attributs démographiques.

3. Preprocessing et Feature Engineering

Le script `preprocessing.py` agit comme la "Raffinerie" du projet, transformant les données bruitées en signaux purs.

3.1. Nettoyage et Imputation

- Les valeurs manquantes (comme l'Age) sont imputées par la **médiane** locale, une méthode statistiquement plus robuste que la moyenne face aux asymétries de distribution.
- Les valeurs aberrantes critiques (ex : `SupportTickets` extravagants ou scores de `Satisfaction` négatifs) sont plafonnées selon les percentiles pour préserver la structure globale des données.

3.2. Feature Engineering

- **Ratios Comportementaux** : Création de variables expertes comme `MonetaryPerDay` pour capturer la dynamique temporelle de l'engagement client.
- **Target Encoding** : Application d'un encodage ciblé sur des variables à haute cardinalité comme `Country`, réduisant drastiquement la dimensionnalité tout en conservant le signal prédictif.
- **Multicolinéarité** : Les caractéristiques dont la corrélation de Pearson dépasse 0.85 sont éliminées pour éviter la redondance et l'instabilité des modèles. Après ces transformations, le dataset final passe de 52 variables brutes à **74 features mathématiquement saines**.

3.3. La Règle d'Or : Zéro "Data Leakage"

La séparation Train/Test (Split 80/20) est effectuée en *stratification* pour garantir la représentativité des 18% de churn dans les deux jeux. De plus, le `StandardScaler` est ajusté (**fit**) **exclusivement sur les données d'entraînement**. Cette rigueur garantit qu'aucune information future ne vient "contaminer" l'apprentissage.

4. Modélisation et Algorithmes

L'intelligence du système repose sur la synergie de trois modèles distincts, entraînés via `train_model.py`.

4.1. Modèle 1 : Segmentation Comportementale (K-Means)

- **Objectif** : Cartographier automatiquement les profils clients (Apprentissage non-supervisé).
- **Méthodologie** : Validation croisée visuelle via la méthode "Elbow" et maximisation du Score de Silhouette.
- **Résultats** : Détection convergente de **k=2** clusters optimaux distincts.
 - *Cluster 0* : Les "Standards" (comportement d'achat modéré et régulier).
 - *Cluster 1* : Les "Hautes Valeurs" (très actifs, forts revenus, cœur de cible).

4.2. Modèle 2 : Détection du Churn (Random Forest)

- **Objectif** : Anticiper le départ d'un client (Classification supervisée).
- **Méthodologie** : Algorithme Random Forest. Au lieu de générer de fausses données (SMOTE), le déséquilibre intrinsèque est neutralisé élégamment via l'hyperparamètre `class_weight='balanced'`.
- **Performances Exceptionnelles** :

- **F1-Score : 0.87**
- **Accuracy : 95.5%**
- **AUC-ROC : 0.99**
- **Interprétabilité (Feature Importance)** : L'analyse des arbres révèle formellement que le **comportement** (Récence, Fréquence, Montant) dicte le churn, balayant l'idée reçue que la démographie serait le facteur clé.

4.3. Modèle 3 : Prévission Financière (Régression Lasso)

- **Objectif** : Évaluer le revenu futur potentiel d'un client (Régression continue).
- **Méthodologie** : Confrontation des algorithmes régularisés Ridge (L2) et Lasso (L1).
- **Résultats** : Le modèle **Lasso** s'impose pour sa capacité intégrée à annuler les variables non pertinentes, créant ainsi un modèle plus léger et direct.
 - **R² (Score d'explication) : 0.81** (Le modèle parvient à expliquer 81% de la variance financière complexe des clients).
 - **RMSE** : ≈ 1995 .

5. Matrice d'Action Business (Insights)

Tout l'intérêt du Machine Learning réside dans l'action. En croisant les résultats algorithmiques (Segmentation $\times$ Prédiction Churn), nous avons modélisé quatre stratégies d'intervention automatisées :

Segment (K-Means)	Prédiction Churn	Stratégie Marketing Proposée
Standard	Fidèle	Upsell : Campagnes classiques et offres croisées.
Haute Valeur	Fidèle	Programme VIP : Intégration au cercle Ambassadeur exclusif.
Standard	Risque	Rétention Proactive : Distribution de bons d'achat ciblés.
Haute Valeur	Risque	Win-Back Prioritaire : Intervention humaine immédiate, offre agressive.

6. Déploiement et Opérationnalisation

L'intégralité du pipeline a été rendue accessible à travers une application web **Flask** (`app/app.py` et `src/predict.py`), comblant ainsi le fossé entre la Data Science et les équipes métiers.

- **Accessibilité** : Une interface utilisateur claire permettant aux conseillers de tester un client.
- **Inférence Temps Réel** : L'API standardise à la volée les nouvelles données et fait appel aux 3 modèles simultanément.
- **Triage Automatisé par Niveaux de Risque** :
 - **Critique (Risque > 45%)** : Alerte rouge, le churn est imminent.
 - **Modéré (Risque > 30%)** : Signal d'alarme, client à surveiller.
 - **Faible (Risque < 30%)** : Clientèle saine et fidèle.

7. Conclusion et Évolutions Futures (MLOps)

Ce projet constitue une validation technique solide sur l'intégralité du spectre data : de l'ingénierie des features complexes à la conception d'API d'inférence, en passant par l'optimisation rigoureuse de modèles prédictifs pénalisés.

Prochaines étapes pour un passage à l'échelle :

1. **Conteneurisation (Docker)** : Isolation de l'environnement Flask pour un déploiement *Serverless* sur le cloud (AWS, Azure, GCP).
2. **Base de Données Opérationnelle** : Remplacement du backend par PostgreSQL pour historiser le comportement et les performances des requêtes.
3. **Continuous Training (CI/CD)** : Implémentation de pipelines GitHub Actions permettant un réentraînement sans intervention humaine.
4. **Détection du "Data Drift"** : Monitoring statistique passif pour alerter les équipes lorsque le profil de la clientèle en direct commence à dévier de nos données d'entraînement.